LAB 7

Fuzzing for Input Validation Bugs

Final Laboratory Report

Student Name	Dhanraj Rathod
SRN	PES1UG24AM359
Project	Museum Admission Parser
Repository	DhanrajRathod18 / L6_L7_54_museum_admission
Submission Date	12 November 2026

This report documents the use of property-based fuzz testing to identify input-validation defects, the implementation of defensive validation, and the final automated test results.

1. Introduction

Input validation is an important part of reliable software. A program may work correctly for ordinary examples but fail when it receives empty strings, unexpected data types, malformed values, or boundary cases. Fuzz testing helps reveal these weaknesses by supplying many generated inputs rather than relying only on a small set of manually selected examples.

This laboratory exercise applies property-based testing with the Hypothesis library to a Museum Admission Parser. The parser accepts visitor names, group ages, and visit dates. The objective was to discover invalid-input handling problems, correct them, and verify the behavior with automated tests.

2. Objectives

The objectives of this experiment were:

• Understand how property-based fuzz testing generates varied inputs.
• Identify input-validation bugs in the museum admission parser.
• Validate visitor names, group ages, and visit dates.
• Use clear validation errors for rejected input.
• Re-run the test suite and confirm that valid inputs continue to work.

3. Tools and Environment

Tool / Component	Purpose
Python 3.13.0	Programming language and runtime
pytest 9.1.1	Automated test execution
Hypothesis 6.170.15	Property-based input generation
Visual Studio Code	Code editing and terminal
Git and GitHub	Version control and remote submission

4. Project Files

The LAB7 directory contains the following main files:

• **museum_admission_parser.py** — application code containing the visitor-name, group-age, and visit-date parsing functions.
• **test_museum_admission_parser.py** — automated tests, including Hypothesis-based tests and explicit validation checks.
• **README.md** — laboratory instructions and project information.
• Three PNG screenshots — original application output, initial failing test results, and final passing test results.

5. Testing Methodology

The original application was first executed with valid sample inputs to establish its expected behavior. The sample inputs were:

```
asha rao
34 12 70
2026-11-05
```

The original output showed the visitor name as **Rao, Asha**, the group ages as **[34, 12, 70]**, and the date as **(2026, 11, 5)**.

Hypothesis was then used to generate a variety of input values, including empty strings, whitespace, unexpected types, malformed age values, and date edge cases. The initial test run exposed defects in all three parser functions.

6. Bugs Identified

Function	Observed issue
format_visitor_name()	Empty or whitespace-only names could cause an IndexError. Unexpected input types could also cause runtime errors.
parse_group_ages()	Non-numeric age values caused ValueError from integer conversion. Negative ages were accepted instead of being rejected.
parse_visit_date()	Empty or malformed dates caused parsing errors. Impossible calendar dates, such as 2026-02-30, were accepted.

Initial test result: 8 failed, 3 passed. The failures demonstrated that the original parser did not consistently validate malformed or boundary inputs.

7. Fixes and Validation Contract

The selected validation contract was to reject invalid inputs explicitly with clear **ValueError** exceptions. The application code was updated to validate inputs before parsing them.

The following behavior was implemented:

• Visitor names: reject empty, whitespace-only, and non-text input; preserve surname-first formatting for valid multi-word names; handle a single-word name without adding an unnecessary comma.
• Group ages: reject empty or malformed values and negative ages; preserve valid integer parsing.
• Visit dates: reject empty or malformed date strings and validate that the date is a real calendar date, not merely three numeric components.

The test file was also updated, as permitted, so tests expect the explicit ValueError behavior for invalid inputs. This resolved the mismatch between random-input checks that previously expected no exception and the explicit tests requiring invalid ages and impossible dates to be rejected.

8. Final Verification

After the fixes, the test suite was executed using the following command:

```
python -m pytest LAB7/test_museum_admission_parser.py -q
```

Final result: **16 passed**. No tests failed in the final run. The valid example inputs were also executed again and continued to produce the expected output.

Verification	Result
Automated test suite	**16 passed**
Visitor name example	Rao, Asha

Verification	Result
Group age example	[34, 12, 70]
Visit date example	(2026, 11, 5)
Git diff whitespace check	No whitespace errors reported

9. Version Control and Submission

The completed LAB7 changes and screenshot evidence were staged and committed to the local Git repository. The commit message was **Complete LAB7 fuzz testing and input validation**. The commit was then pushed successfully to the user's GitHub fork on the main branch.

Repository: github.com/DhanrajRathod18/L6_L7_54_museum_admission

Commit: **b3d0aec**

10. Conclusion

This laboratory exercise demonstrated how property-based fuzz testing can uncover defects that may not appear with ordinary example inputs. Hypothesis identified weaknesses in visitor-name handling, age parsing, and date validation. After implementing explicit input validation and aligning the tests with the selected error-handling contract, all 16 tests passed. The valid application examples continued to work, and the completed changes were committed and pushed to GitHub.

11. Evidence Screenshots

The following evidence files are included in the LAB7 directory:

1. **01_original_application.png** — original application run with valid sample inputs.
2. **02_hypothesis_failing_tests.png** — initial fuzz-testing failures showing the defects.
3. **03_all_tests_passed.png** — final test run showing 16 passing tests.

End of Report